```
  GNU nano 7.2                                                    co
#include <stdio.h>
#include <string.h>

int main() {
    char command[100];

    while (1) {
        printf("myshell> ");
        fgets(command, sizeof(command), stdin);

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0) {
            printf("Exiting shell...\n");
            break;
        }

        if (strlen(command) > 0) {
            printf("You entered: %s\n", command);
        }
    }
```

```
bhavan@LAPTOP-14JC0IVG: ~$ This message is shown once a day. To disable it please
create the /home/bhavan/.hushlogin file.
bhavan@LAPTOP-14JC0IVG: ~$ nano copy.c
bhavan@LAPTOP-14JC0IVG: ~$
```

```
OSLab  OSSP  Week3  copy.c  process  process.c  process.c.save  process.c.save.1
bhavan@LAPTOP-14JC0IVG:~$ cat File1.txt
Hello, this is File 1.
This content will be copied to File 2.
bhavan@LAPTOP-14JC0IVG:~$ cat File2.txt
cat: File2.txt: No such file or directory
bhavan@LAPTOP-14JC0IVG:~$ Hello, this is File 1.
bash: Hello,: command not found
bhavan@LAPTOP-14JCC0IVG:~$ This content will be copied to File 2.
bash: This: command not found
bhavan@LAPTOP-14JC0IVG:~$ gcc copy.c -o copy
bhavan@LAPTOP-14JC0IVG:~$ ./copy
Hello, this is File 1.
This content will be copied to File 2.
bhavan@LAPTOP-14JC0IVG:~$ cat File2.txt
Hello, this is File 1.
This content will be copied to File 2.
bhavan@LAPTOP-14JC0IVG: ~$
```

Description:

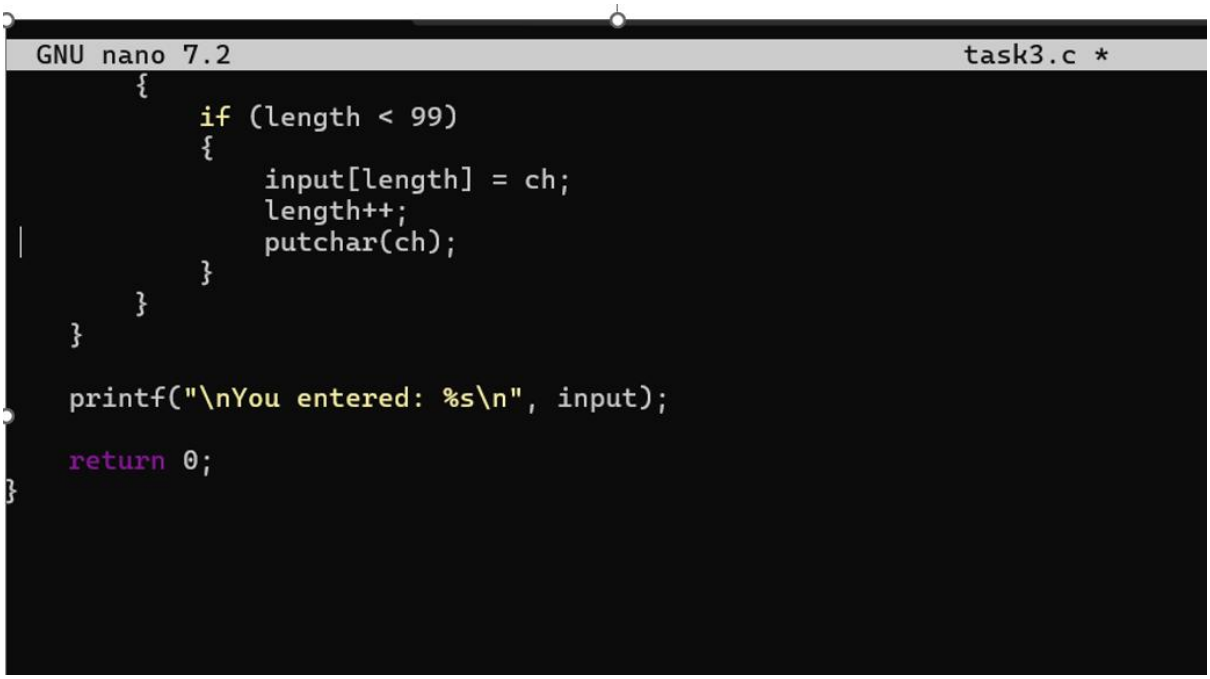
In this task, we use the system calls open(), read(), write(), and close(). First, we open File1.txt

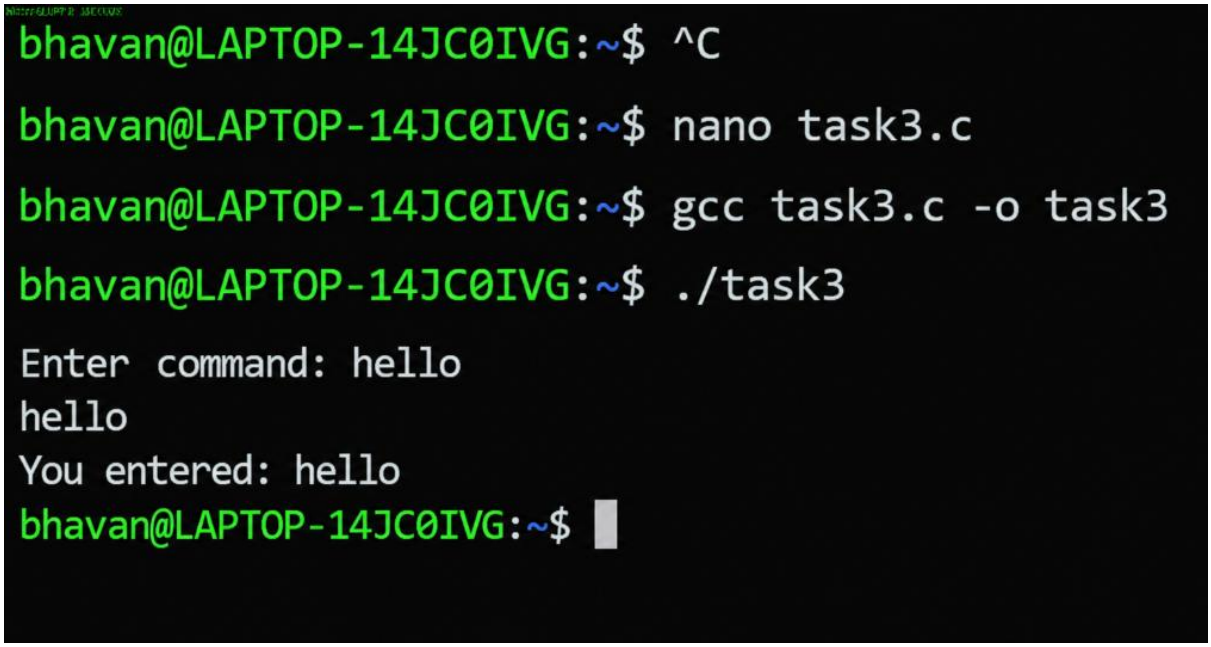
for reading. Then, we read the content from the file and write it into File2.txt. Finally, we close both files.

The main system calls used are open(), read(), write(), and close().

Result:
The contents of File1.txt were successfully copied to File2.txt.

TASK 3





o capture keyboard input and handle the Backspace and Enter keys.

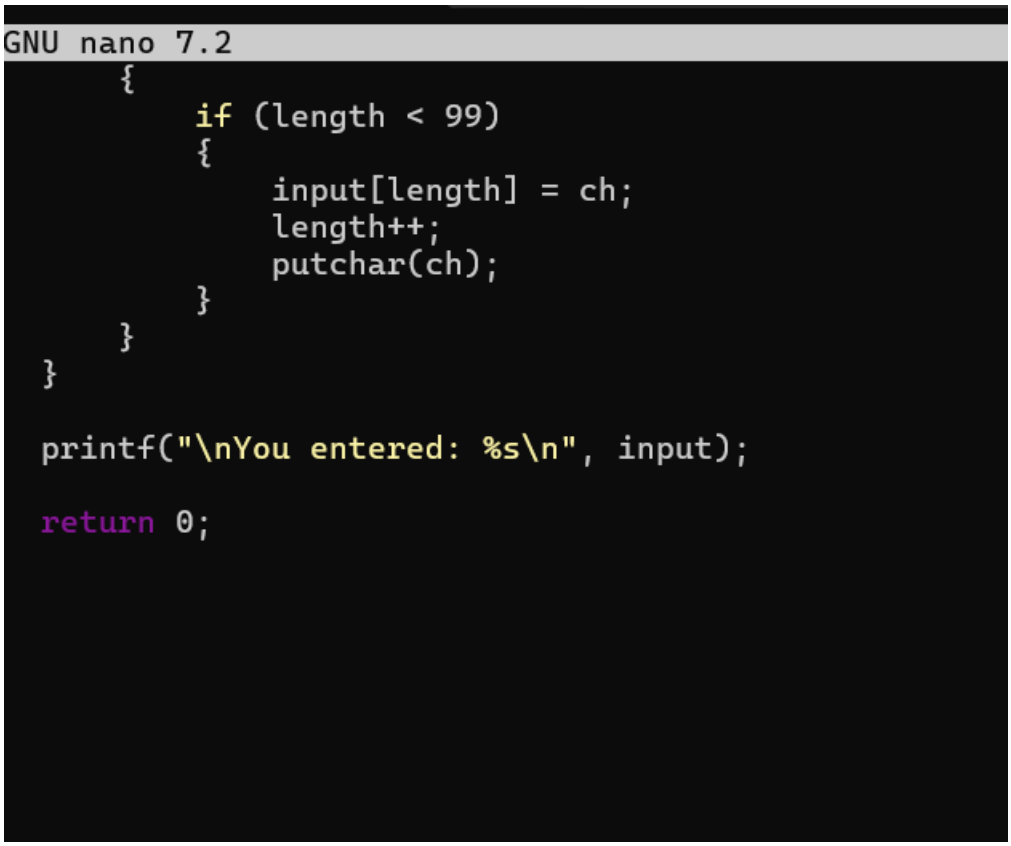
Theory:
In this task, the program takes input from the keyboard and stores it in an input buffer. Each character entered by the user is stored in the buffer.

The **Backspace key** is used to remove the last character entered. The **Enter key** is used to finish the input. After pressing Enter, the program processes the complete command and displays the entered text.

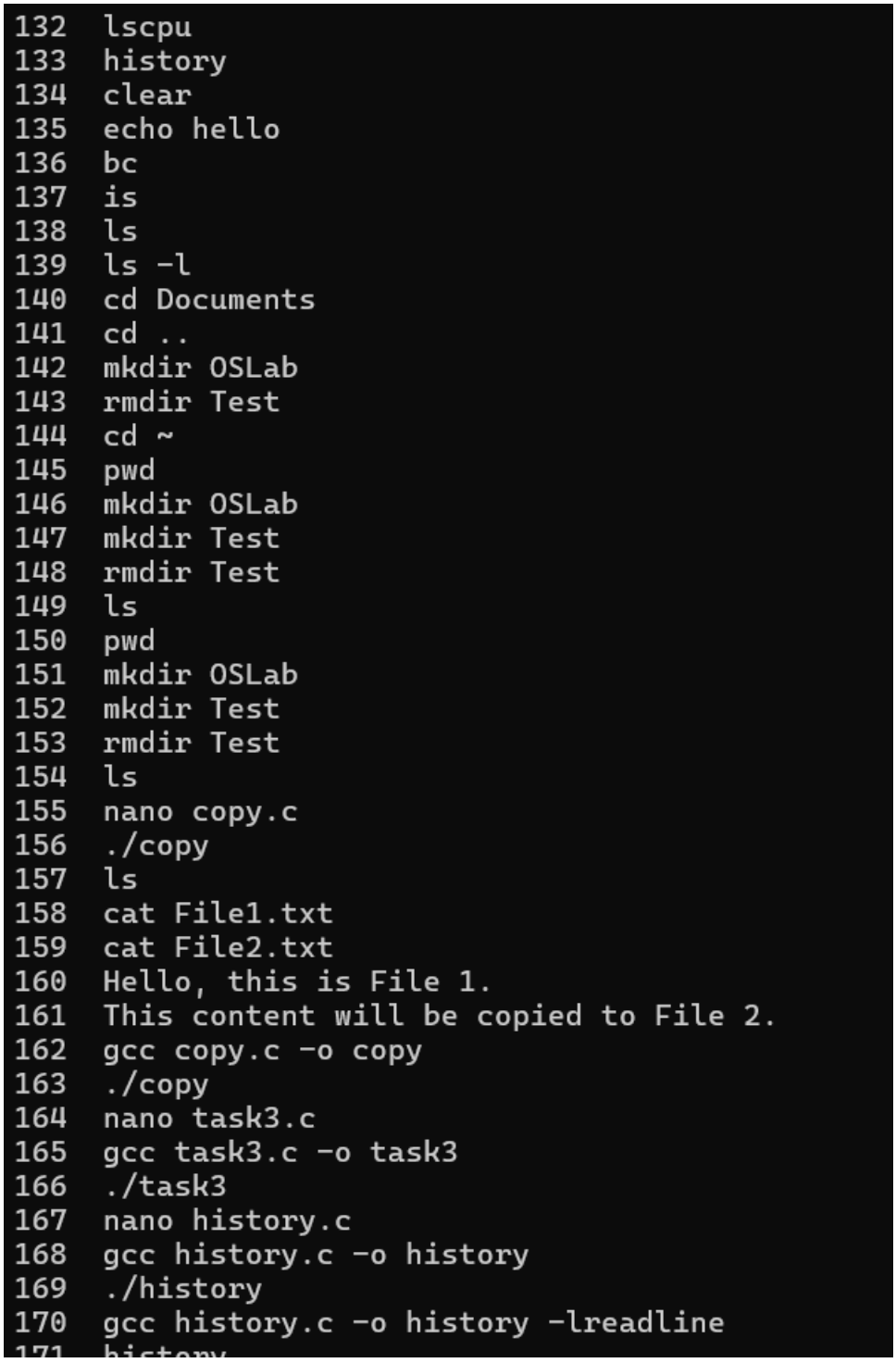
This task helps us understand how keyboard input is captured, how the input buffer is managed, and how multi-character commands are handled.

Result:
The keyboard input was successfully captured, and the Backspace and Enter keys were handled

TASK 4:



```
 1  cfffcvgvgvvvvvvvvv
 2  clear
 3  sudo apt update
 4  sudo apt install build-essential installing:
 5  gcc --version
 6  gcc process . c -o process
 7  ./process
 8  clear
 9  mkdir OSSP
10  cd OSSP
11  git init
12  mkdir src include obj bin docs tests scripts assets
13  touch .gitignore README.md LICENSE Makefile
14  touch src/main.c
15  touch src/shell.c
16  touch src/parser.c
17  touch src/executor.c
18  touch src/builtin.c
19  touch include/shell.h
20  touch include/parser.h
21  touch include/executor.h
22  touch include/builtin.h
23  touch docs/design.md
24  touch docs/report.pdf
25  touch tests/test_parser.c
26  touch tests/test_executor.c
27  touch scripts/run.sh
28  touch assets/architecture.png
29  tree
30  sudo apt install tree
31  tree
32  sudo apt install tree
33  sudo apt install build-essential
34  gcc --version
35  nano process.c
36  sudo apt install tree
37  sudo apt install build-essential
38  nano process.c
39  pwd
40  clear
```



To implement command history in a simple shell and allow the user to access previously entered commands.

Theory

In this task, the program stores the commands entered by the user in a history buffer. The user can move through the previous commands using the appropriate keys and can edit the command before executing it.

The task also focuses on handling the input buffer correctly while moving between previous and next commands. This helps us understand how command history works in a shell.

Result

The command history was implemented successfully, and previously entered commands could be accessed and handled through the input buffer.

Task 5

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Node {
    char *data;
    struct Node *next;
};

int main()
{
    int size = 10;
    char *buffer;
    struct Node *head = NULL;
    struct Node *newNode;

    /* Dynamic buffer allocation */
    buffer = (char *)malloc(size * sizeof(char));

    if (buffer == NULL)
    {
        printf("Memory allocation failed\n");
        return 1;
    }
```

```
bhavan@LAPTOP-14JC0IVG:~$ nano memory.c
bhavan@LAPTOP-14JC0IVG:~$ gcc -g memory.c -o memory
bhavan@LAPTOP-14JC0IVG:~$ ./memory
Enter a string: hello
Entered text: hello

Linked list data: hello

Memory released successfully.
bhavan@LAPTOP-14JC0IVG:~$ valgrind --leak-check=full ./memory
Command 'valgrind' not found, but can be installed with:
sudo snap install valgrind  # version 3.26.0,, or
sudo apt  install valgrind  # version 1:3.22.0-0ubuntu2
See 'snap info valgrind' for additional versions.
bhavan@LAPTOP-14JC0IVG:~$ sudo apt update
[sudo] password for bhavan:
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:5 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [930 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1190 kB]
Get:7 http://archive.ubuntu.com/ubuntu noble-updates/main Translation-en [282 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [201 kB]
Get:9 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [181 kB]
Get:10 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1683 kB]
Get:11 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [46.4 kB]
Get:12 http://archive.ubuntu.com/ubuntu noble-updates/universe Translation-en [335 kB]
Get:13 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1200 kB]
Get:14 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [388 kB]
Get:15 http://archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Packages [1424 kB]
Get:16 http://archive.ubuntu.com/ubuntu noble-updates/restricted Translationn-en [323 kB]
Get:17 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [76.2 kB]
Get:18 http://archive.ubuntu.com/ubuntu noble-updates/multiverse Translation-en [12.6 kB]
```

```
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following package was automatically installed and is no longer required:
  libglapi-mesa
Use 'sudo apt autoremove' to remove it.
The following additional packages will be installed:
  gdb libbabeltrace1 libc6-dbg libdebuginfod-common libdebuginfod1t64 libipt2 libsource-highlight-common libsource-highlight4t64
Suggested packages:
  gdb-doc gdbserver valgrind-dbg valgrind-mpi kcachegrind alleyoop valkyrie
The following NEW packages will be installed:
  gdb libbabeltrace1 libc6-dbg libdebuginfod-common libdebuginfod1t64 libipt2 libsource-highlight-common libsource-highlight4t64 valgrind
0 upgraded, 9 newly installed, 0 to remove and 15 not upgraded.
Need to get 27.0 MB of archives.
After this operation, 103 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libdebuginfod-common all 0.190-1.1ubuntu0.1 [14.6 kB]
Get:2 http://archive.ubuntu.com/ubuntu noble/main amd64 libbabeltrace1 amd64 1.5.11-3build3 [164 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libdebuginfod1t64 amd64 0.190-1.1ubuntu0.1 [17.1 kB]
```

```
bhavan@LAPTOP-14JC0IVG:~$ ./memory
Enter a string: hello
Entered text: hello

Linked list data: hello

Memory released successfully.
bhavan@LAPTOP-14JC0IVG:~$ valgrind --leak-check=full ./memory
Command 'valgrind' not found, but can be installed with:
sudo snap install valgrind  # version 3.26.0,, or
sudo apt  install valgrind  # version 1:3.22.0-0ubuntu2
See 'snap info valgrind' for additional versions.
bhavan@LAPTOP-14JC00IVG:~$ sudo apt update
[sudo] password for bhavan:
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:5 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [930 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1190 kB]
Get:7 http://archive.ubuntu.com/ubuntu noble-updates/main Translation-en [282 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [201 kB]
Get:9 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [181 kB]
Get:10 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1683 kB]
Get:11 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [46.4 kB]
Get:12 http://archive.ubuntu.com/ubuntu noble-updates/universe Translation-en [335 kB]
Get:13 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1200 kB]
Get:14 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [388 kB]
Get:15 http://archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Packages [1424 kB]
Get:16 http://archive.ubuntu.com/ubuntu noble-updates/restricted Translationn-en [323 kB]
Get:17 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [76.2 kB]
Get:18 http://archive.ubuntu.com/ubuntu noble-updates/multiverse Translation-en [12.6 kB]
Get:19 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [1320 kB]
Get:20 http://archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
Get:21 http://archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [5772 B]
Get:22 http://security.ubuntu.com/ubuntu noble-security/restricted Translation-en [302 kB]
Get:23 http://archive.ubuntu.com/ubuntu noble-backports/universe amd64 Components [12.6 kB]
Fetched 10.3 MB in 7s (14777 kB/s)
Reading package lists... Done
```

```
bhavan@LAPTOP-14JC0IVG:~$ valgrind --leak-check=full ./memory
==10188== Memcheck, a memory error detector
==10188== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==10188== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==10188== Command: ./memory

Enter a string: hello
Entered text: hello

Linked list data: hello

Memory released successfully
==10188==
==10188== HEAP SUMMARY:
==10188==     in use at exit: 0 bytes in 0 blocks
==10188==   total heap usage: 6 allocs, 6 frees, 2,181 bytes allocated
==10188==
==10188== All heap blocks were freed -- no leaks are possible
==10188==
==10188== For lists of detected and suppressed errors, rerun with: -s
==10188== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
bhavan@LAPTOP-14JC0IVG:~$
```

To allocate memory dynamically, resize buffers, manage linked lists, release memory correctly, and verify memory usage using Valgrind.

Theory:
In this task, the program uses dynamic memory allocation to create buffers when memory is needed. The allocated memory can be resized according to the requirement. The program also manages data using a linked list.

Proper memory management is important to avoid buffer overflow and memory leaks. After completing the required operations, the allocated memory is released using the appropriate functions.

Finally, the program is checked using the **Valgrind** tool to verify that memory is being used correctly and that there are no memory leaks